LAMP: Latent Associative Memory Programming in a Frozen Toy Transformer

Technical Report: November 13, 2026

WILLIAM TAGGART, Virginia Tech, USA

We introduce LAMP (Latent Associative Memory Programming), an inspectable memory adaptor that changes selected answers of a frozen transformer by directly writing recognition keys and activation differences into two parameter matrices. A writer captures a source question's hidden state and selects a donor state that the frozen model already decodes into a desired answer. It stores their difference, then conditionally adds that difference to a matching state during inference. Memory writes require forward passes but no gradient-based optimization. A two-block, 77,568-parameter question-and-answer model is augmented with 2,048 memory weights across 16 slots. A counterfactual Texas-capital edit succeeds on the exact source and one of two held-out paraphrases, with no activation on 34 unrelated held-out questions. These results establish a restricted demonstration of selective activation editing, not a general episodic-memory system. A browser implementation exposes the memory weights for direct manipulation.

Additional Key Words and Phrases: associative memory, activation intervention, frozen transformer, forward-only writing

1 MOTIVATION AND SCOPE

A pretrained model's behavior can be changed by altering its numerical computation rather than appending instructions to its context. We investigate a deliberately small version of this idea: freeze the learned base weights, add an initially inactive memory module, and provide an explicit writer that programs selected associations.

The illustrative task is to make "What is the capital of Texas?" produce "Sacramento" while preserving unrelated answers. This is an intentional counterfactual override, not a factual correction. Users can inspect the memory values, change entries, disable slots, and compare original and edited predictions.

Our contribution is an educational implementation combining latent-state matching and donor-based activation transplantation. We do not claim that associative weight storage, key–value adaptors, or activation interventions are new. The base is trained once; subsequent memory writes are forward-only.

2 FROZEN MODEL AND INSERTION POINT

The base is a causal transformer with two blocks, hidden width $d = 64$, four attention heads, and a two-layer GELU feed-forward network of width 128 in each block. It has 58 vocabulary tokens, learned positional embeddings for 48 positions, and an untied output head. It predicts one answer token rather than generating a conversation.

Inputs include a beginning marker and a final <answer> marker. Let $h(x) \in \mathbb{R}^{1\times d}$ be the residual state at that final marker, immediately after the second block's attention and residual addition. Memory modifies h *before* the second block's feed-forward LayerNorm, affecting both its feed-forward input and residual path. There is no attention after this point. The remaining computation is

$$u = h' + \mathrm{FFN}_2(\mathrm{LN}_2(h')), \tag{1}$$

$$F_\theta(h') = \mathrm{LN}_{\mathrm{final}}(u)W_{\mathrm{out}}^\top, \tag{2}$$

where $W_{\mathrm{out}} \in \mathbb{R}^{58\times 64}$ and all base parameters θ remain fixed.

Author's address: William Taggart, Modern and Classical Languages and Literatures, Virginia Tech, Blacksburg, Virginia, USA, wtaggart@vt.edu.

Component	Scalar parameters
Token and position embeddings	6,784
Two transformer blocks	66,944
Final LayerNorm and output head	3,840
Frozen base total	77,568
Memory keys and deltas	2,048

3 MEMORY OPERATIONS

3.1 Selective retrieval

The adaptor contains $K, D \in \mathbb{R}^{m \times d}$, with $m = 16$ and $d = 64$, plus an enabled flag b_i and threshold τ_i for each slot. Both matrices start at zero and every slot starts disabled, making the initial adaptor exactly inactive.

Given a query state h, compute

$$q = \frac{h}{\max(\|h\|_2, \epsilon)}, \qquad s = qK^\top \in \mathbb{R}^{1 \times m}.$$

Let $E = \{i : b_i = 1,\ s_i \geq \tau_i\}$. If E is nonempty, select $j = \arg\max_{i \in E} s_i$, breaking ties by the lowest index, and set $h' = h + D_j$. Otherwise $h' = h$. The default threshold is 0.95. Writer-created keys have unit length, making scores cosine similarities. Manual edits do not renormalize keys; their scores are normalized-query dot products and may exceed one.

Matching costs $O(md)$, followed by an $O(d)$ addition. A conceptual one-hot selector $a \in \mathbb{R}^{1 \times m}$ gives $h' = h + aD$; the implementation gathers the selected row directly.

3.2 Forward-only donor writer

For a source question x_s and desired answer token $y^\star$, compute $h_s = h(x_s)$ with memory disabled. Run a fixed set P of 18 canonical probe questions through the frozen model. Among probes whose argmax prediction equals $y^\star$, choose the donor with greatest log probability for that token:

$$x_t = \arg \max_{\substack{x \in P \\ \arg\max F_\theta(h(x)) = y^\star}} \log \operatorname{softmax}(F_\theta(h(x)))_{y^\star}.$$

Selection uses predictions, not probe gold labels; however, P is deliberately constructed for this restricted domain. If no probe predicts the desired answer, the write is rejected. Otherwise write

$$K_i = \frac{h_s}{\|h_s\|_2}, \qquad D_i = h(x_t) - h_s, \qquad b_i = 1, \quad \tau_i = 0.95.$$

Each write changes at most 128 floating-point weights, plus metadata. An enabled key with dot product above $1 - 10^{-5}$ against the normalized source is reused; otherwise the first disabled slot is chosen. A full memory requires clearing or disabling a slot. This replacement rule assumes normal writer-created keys; unrestricted manual edits can alter its behavior.

For the exact source, if its programmed slot wins retrieval,

$$h_s + D_i = h(x_t).$$

Thus the remaining deterministic computation reproduces the donor's full logit vector, up to numerical error. Placement after the final attention operation is essential to this simple single-state argument. Later cross-token attention would introduce dependence on other token states.

For a different question with state $h_s + e$, retrieval instead gives $h(x_t) + e$. Similarity-based activation therefore does not guarantee the requested answer: retrieval and downstream decoding are separate requirements.

4 IMPLEMENTATION AND PRELIMINARY EVALUATION

The base is trained on 18 facts about capitals, largest cities, animal color universals, and mammal categories. Five templates per fact yield 90 training questions; two other templates yield 36 held-out questions. Held-out templates contribute vocabulary to the tokenizer, but not supervised training pairs. This evaluates wording variation over known facts, not unseen factual knowledge.

Training uses seed 17, AdamW with learning rate 0.003 and weight decay 0.01, batches of 48 sampled with replacement, 600 steps, and gradient clipping at norm 1. The checkpoint is then frozen. The following results come from the saved CPU demonstration run; they are not averages over seeds.

Measurement	Observed result
Base training accuracy	90/90
Base held-out accuracy	33/36
Exact Texas → Sacramento override	Succeeds
Target held-out paraphrases changed	1/2
Unrelated held-out memory activations	0/34
Blank adaptor vs. base logits	Identical
Base parameters after memory writes	Unchanged

The successful target paraphrase has similarity 0.9842; the unsuccessful one has similarity 0.8439, below the 0.95 threshold. All 34 unrelated held-out questions retain their base behavior, including existing base errors. This supports locality only on this small test set.

Functional checks also demonstrate simultaneous Texas → Sacramento and cats-black → yes associations; replacement of Texas with Paris in the same slot; persistence after reload; and restoration of baseline behavior after clearing memory. These checks do not establish capacity at 16 occupied slots.

A dependency-free JavaScript port executes the actual transformer from exported FP32 weights. Across all 126 training and held-out questions, its base argmax predictions match Python, with maximum observed absolute logit difference approximately 3.02×10^{-6}. JavaScript arithmetic and an approximate error function for GELU introduce numerical differences. Raw base weights occupy 310,272 bytes; the memory matrices require 8,192 bytes in FP32, excluding metadata and serialization overhead.

Python checkpoints and browser-local storage preserve memories across sessions. The browser interface provides numeric import/export, raw weight editing, and undo. Persistence is explicit storage, not automatic learning during inference.

5 RELATIONSHIP TO PRIOR WORK

GRACE uses discrete key–value adaptors to selectively edit a frozen model and is a close architectural precedent [2]. Our toy implementation constructs a donor difference directly rather than optimizing an edit value. ROME changes

existing feed-forward weights to edit factual associations [3]; our original base matrices remain unchanged. Contrastive activation addition steers behavior with activation differences [4]; our delta is source-specific and gated by a stored key. Fast-weight programmers relate dynamic matrix updates to associative memory [5]. Titans develops neural memory that learns at test time [1], a broader setting than the present explicit writer.

6 LIMITATIONS AND NEXT EXPERIMENTS

This memory is an explicit latent association table with one slot per edit. Storing its arrays as parameters does not demonstrate dense consolidation of many experiences into shared base weights. Neither readable question strings nor answer IDs are stored in these arrays, but fixed probe questions and a tokenizer remain part of the application.

The writer cannot introduce new vocabulary or guarantee a donor for every token. The model supports one-token answers, explicit writes, and limited paraphrase generalization. It does not record every forward pass, reconstruct past questions, or provide conversational episodic recall. Arbitrary raw edits have no guaranteed semantic interpretation. Multiple eligible slots compete through hard selection, and thresholds trade recall against unintended activation.

Next experiments should evaluate multiple training seeds, all occupied slots, threshold sweeps, and harder negative queries. Baselines should include nearest-neighbor output substitution and optimized adaptor values. Longer answers and earlier insertion points require additional mechanisms and fresh evaluation.

7 CONCLUSION

A frozen toy transformer can be selectively redirected by storing a normalized recognition key and a donor-minus-source hidden-state difference. The construction gives an exact-source guarantee under successful retrieval and a transparent interface for inspecting memory weights. Its principal value is an executable educational demonstration of forward-only associative editing, with bounded claims about memory and generalization.

AI-USE DISCLOSURE

This project was created with assistance from GPT-6 Astra and Codex. AI tools supported software development, technical exploration, and drafting and revision of the report. William Taggart is responsible for the research decisions and final software and documentation. AI assistance does not constitute independent validation of the implementation or its scientific claims.

REFERENCES

[1] Behrouz, Zhong, and Mirrokni. 2025. Titans: Learning to Memorize at Test Time. https://arxiv.org/abs/2501.00663 arXiv:2501.00663.
[2] Hartvigsen et al. 2023. Aging with GRACE: Lifelong Model Editing with Discrete Key-Value Adaptors. https://arxiv.org/abs/2211.11031 NeurIPS.
[3] Meng et al. 2022. Locating and Editing Factual Associations in GPT. https://arxiv.org/abs/2202.05262 NeurIPS.
[4] Rimsky et al. 2024. Steering Llama 2 via Contrastive Activation Addition. https://aclanthology.org/2024.acl-long.828/ ACL.
[5] Schlag, Irie, and Schmidhuber. 2021. Linear Transformers Are Secretly Fast Weight Programmers. https://arxiv.org/abs/2102.11174 ICML.

SUGGESTED CITATION

William Taggart. 2026. LAMP: Latent Associative Memory Programming in a Frozen Toy Transformer. Technical report. Modern and Classical Languages and Literatures, Virginia Tech. ACM manuscript edition, November 13, 2026. https://github.com/FL2744/LAMP